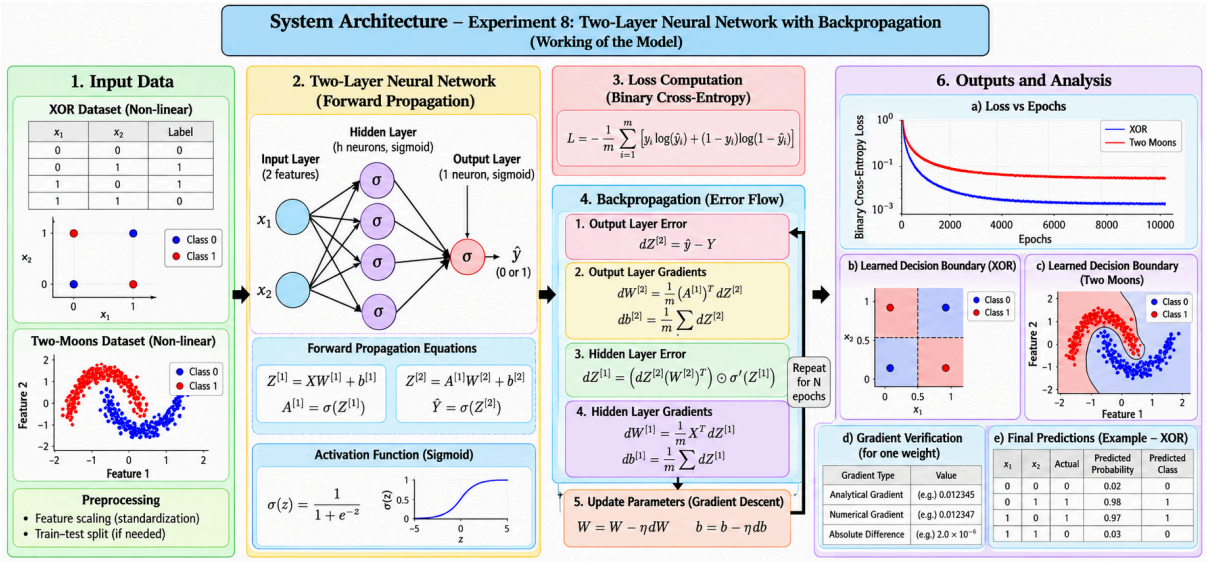
Experiment 8: Two-Layer Neural Network with Backpropagation

1. Aim

To implement a two-layer neural network from scratch using NumPy, perform forward propagation using sigmoid activation functions, and train the network using backpropagation and gradient descent. To visualize the loss against epochs, plot the nonlinear decision boundary for XOR data, and verify the computed gradients numerically.



2. Mathematical Formulas

1. Forward Propagation

Hidden layer:

$$\boxed{Z^{[1]} = XW^{[1]} + b^{[1]}}$$

$$\boxed{A^{[1]} = \sigma(Z^{[1]})}$$

Output layer:

$$\boxed{Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}}$$

$$\boxed{\hat{Y} = \sigma(Z^{[2]})}$$

2. Sigmoid Activation Function

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

$$\boxed{\sigma'(z) = \sigma(z)(1 - \sigma(z))}$$

3. Binary Cross-Entropy Loss

$$\boxed{L = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]}$$

4. Backpropagation

Output layer error:

$$\boxed{dZ^{[2]} = \hat{Y} - Y}$$

Output layer gradients:

$$\boxed{dW^{[2]} = \frac{1}{m}(A^{[1]})^T dZ^{[2]}}$$

$$\boxed{db^{[2]} = \frac{1}{m}\sum dZ^{[2]}}$$

Hidden layer error:

$$\boxed{dZ^{[1]} = (dZ^{[2]}(W^{[2]})^T) \odot \sigma'(Z^{[1]})}$$

Hidden layer gradients:

$$\boxed{dW^{[1]} = \frac{1}{m}X^T dZ^{[1]}}$$

$$\boxed{db^{[1]} = \frac{1}{m}\sum dZ^{[1]}}$$

5. Gradient Descent Update

$$\boxed{W_{\text{new}} = W_{\text{old}} - \eta\, dW}$$

$$\boxed{b_{\text{new}} = b_{\text{old}} - \eta\, db}$$

where η is the learning rate.

6. Numerical Gradient Verification

$$\boxed{\frac{\partial L}{\partial w} \approx \frac{L(w+\epsilon) - L(w-\epsilon)}{2\epsilon}}$$

CODE

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# ========================================
# 1. ACTIVATION AND LOSS FUNCTIONS
# ========================================

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y, pred):
    pred = np.clip(pred, 1e-8, 1 - 1e-8)
    return -np.mean(
        y * np.log(pred) + (1 - y) * np.log(1 - pred)
    )

# ========================================
# 2. INITIALIZE TWO-LAYER NEURAL NETWORK
# ========================================

def initialize_network(input_size, hidden_size):
    return {
        "W1": np.random.randn(input_size, hidden_size) * 0.5,
        "b1": np.zeros((1, hidden_size)),
        "W2": np.random.randn(hidden_size, 1) * 0.5,
        "b2": np.zeros((1, 1))
    }

# ========================================
# 3. FORWARD PROPAGATION
# ========================================

def forward(X, params):
    Z1 = X @ params["W1"] + params["b1"]
    A1 = sigmoid(Z1)

    Z2 = A1 @ params["W2"] + params["b2"]
    A2 = sigmoid(Z2)

    cache = {"Z1": Z1, "A1": A1, "Z2": Z2, "A2": A2}
    return A2, cache

# ========================================
# 4. BACKPROPAGATION
# ========================================

def backward(X, y, params, cache):
    m = len(X)
    A1 = cache["A1"]
    A2 = cache["A2"]

    dZ2 = A2 - y
    dW2 = (A1.T @ dZ2) / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m
```

```
    dZ1 = (dZ2 @ params["W2"].T) * A1 * (1 - A1)
    dW1 = (X.T @ dZ1) / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m

    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}

# ========================================
# 5. TRAIN THE NETWORK
# ========================================

def train_network(X, y, hidden_size=8, lr=1.0, epochs=5000):
    params = initialize_network(X.shape[1], hidden_size)
    losses = []

    for epoch in range(epochs):
        pred, cache = forward(X, params)
        loss = binary_cross_entropy(y, pred)
        losses.append(loss)

        grads = backward(X, y, params, cache)

        for key in params:
            params[key] -= lr * grads[key]

    return params, losses

# ========================================
# 6. XOR DATASET
# ========================================

X_xor = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

y_xor = np.array([[0], [1], [1], [0]], dtype=float)

np.random.seed(42)
xor_params, xor_losses = train_network(
    X_xor, y_xor, hidden_size=4, lr=1.0, epochs=10000
)

xor_pred, _ = forward(X_xor, xor_params)

print("XOR Results")
print("Inputs:\n", X_xor)
print("Actual outputs:", y_xor.ravel())
print("Predicted probabilities:", np.round(xor_pred.ravel(), 4))
print("Predicted classes:", (xor_pred >= 0.5).astype(int).ravel())
print("Final XOR loss:", round(xor_losses[-1], 6))

# Plot XOR loss
plt.figure(figsize=(7, 4))
plt.plot(xor_losses)
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("XOR: Loss vs Epochs")
plt.grid(True)
```

```
plt.show()

# ========================================
# 7. TWO-MOONS DATASET
# ========================================

X_moons, y_moons = make_moons(
    n_samples=300, noise=0.20, random_state=42
)
y_moons = y_moons.reshape(-1, 1)

scaler = StandardScaler()
X_moons = scaler.fit_transform(X_moons)

np.random.seed(42)
moon_params, moon_losses = train_network(
    X_moons, y_moons, hidden_size=8, lr=0.5, epochs=5000
)

moon_pred, _ = forward(X_moons, moon_params)
moon_classes = (moon_pred >= 0.5).astype(int)
accuracy = np.mean(moon_classes == y_moons) * 100

print("\nTwo-Moons Results")
print("Final loss:", round(moon_losses[-1], 4))
print("Training accuracy:", round(accuracy, 2), "%")

# Plot two-moons loss
plt.figure(figsize=(7, 4))
plt.plot(moon_losses)
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Two Moons: Loss vs Epochs")
plt.grid(True)
plt.show()

# ========================================
# 8. PLOT NONLINEAR DECISION BOUNDARY
# ========================================

def plot_boundary(X, y, params, title):
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 250),
        np.linspace(y_min, y_max, 250)
    )

    grid = np.c_[xx.ravel(), yy.ravel()]
    probs, _ = forward(grid, params)
    probs = probs.reshape(xx.shape)

    plt.figure(figsize=(7, 5))
    plt.contourf(xx, yy, probs, levels=20, cmap="coolwarm", alpha=0.6)
    plt.contour(xx, yy, probs, levels=[0.5], colors="black")
    plt.scatter(
        X[:, 0], X[:, 1], c=y.ravel(),
        cmap="coolwarm", edgecolors="k"
    )
    plt.xlabel("Feature 1")
```

```
    plt.ylabel("Feature 2")
    plt.title(title)
    plt.show()

plot_boundary(X_xor, y_xor, xor_params, "XOR Decision Boundary")
plot_boundary(X_moons, y_moons, moon_params, "Two-Moons Decision Boundary")

# ========================================
# 9. NUMERICAL GRADIENT VERIFICATION
# ========================================

# Check one weight of the trained XOR network
pred, cache = forward(X_xor, xor_params)
grads = backward(X_xor, y_xor, xor_params, cache)

epsilon = 1e-5
original = xor_params["W1"][0, 0]

xor_params["W1"][0, 0] = original + epsilon
loss_plus = binary_cross_entropy(
    y_xor, forward(X_xor, xor_params)[0]
)

xor_params["W1"][0, 0] = original - epsilon
loss_minus = binary_cross_entropy(
    y_xor, forward(X_xor, xor_params)[0]
)

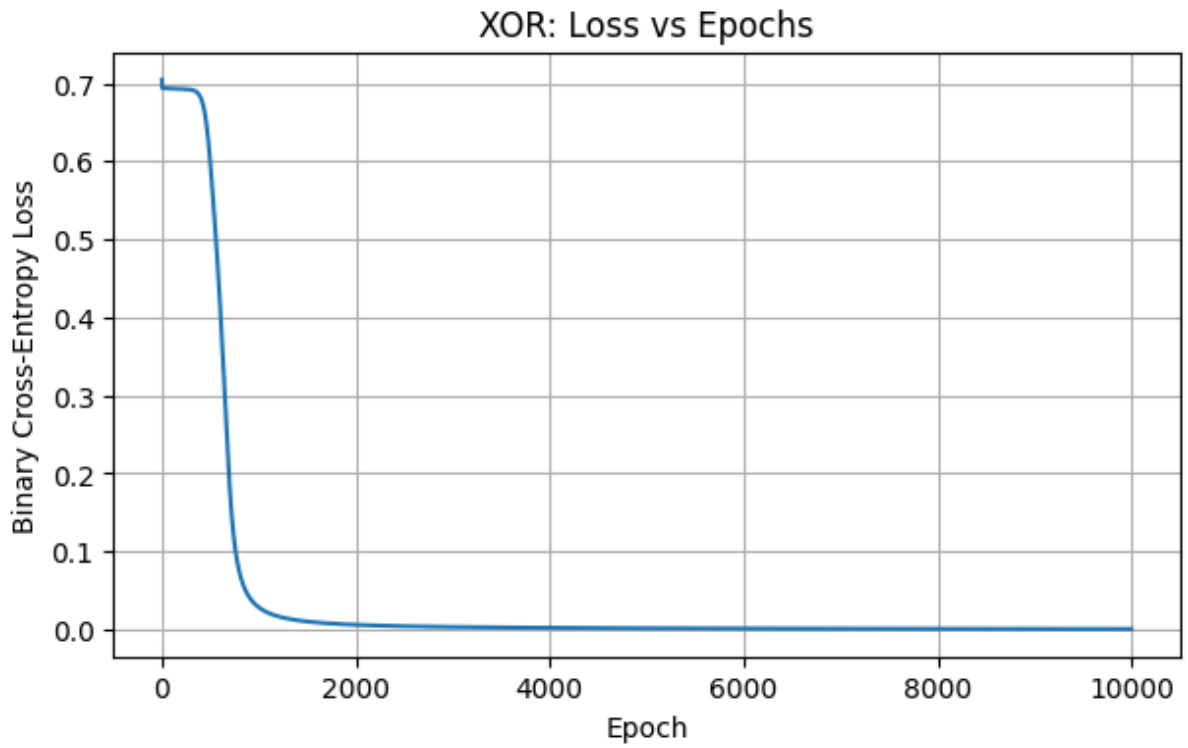
xor_params["W1"][0, 0] = original

numerical_grad = (loss_plus - loss_minus) / (2 * epsilon)
analytical_grad = grads["W1"][0, 0]

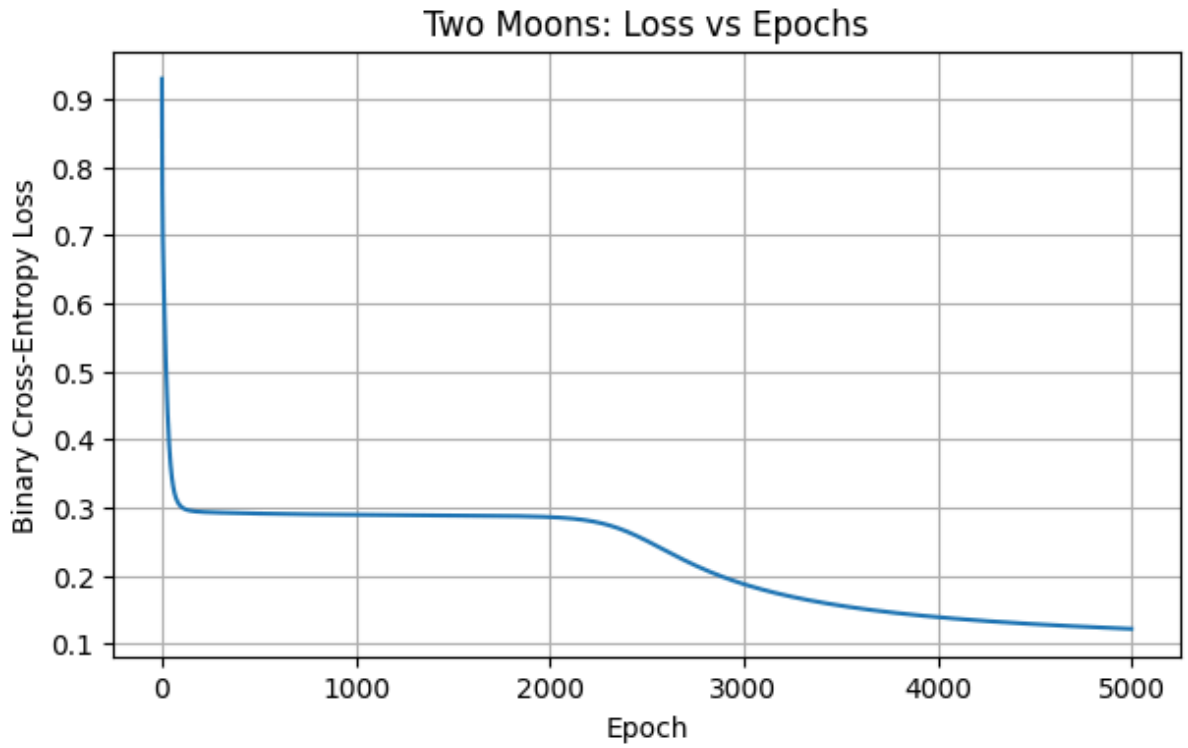
print("\nGradient Verification")
print("Analytical gradient:", analytical_grad)
print("Numerical gradient:", numerical_grad)
print(
    "Absolute difference:",
    abs(analytical_grad - numerical_grad)
)
```

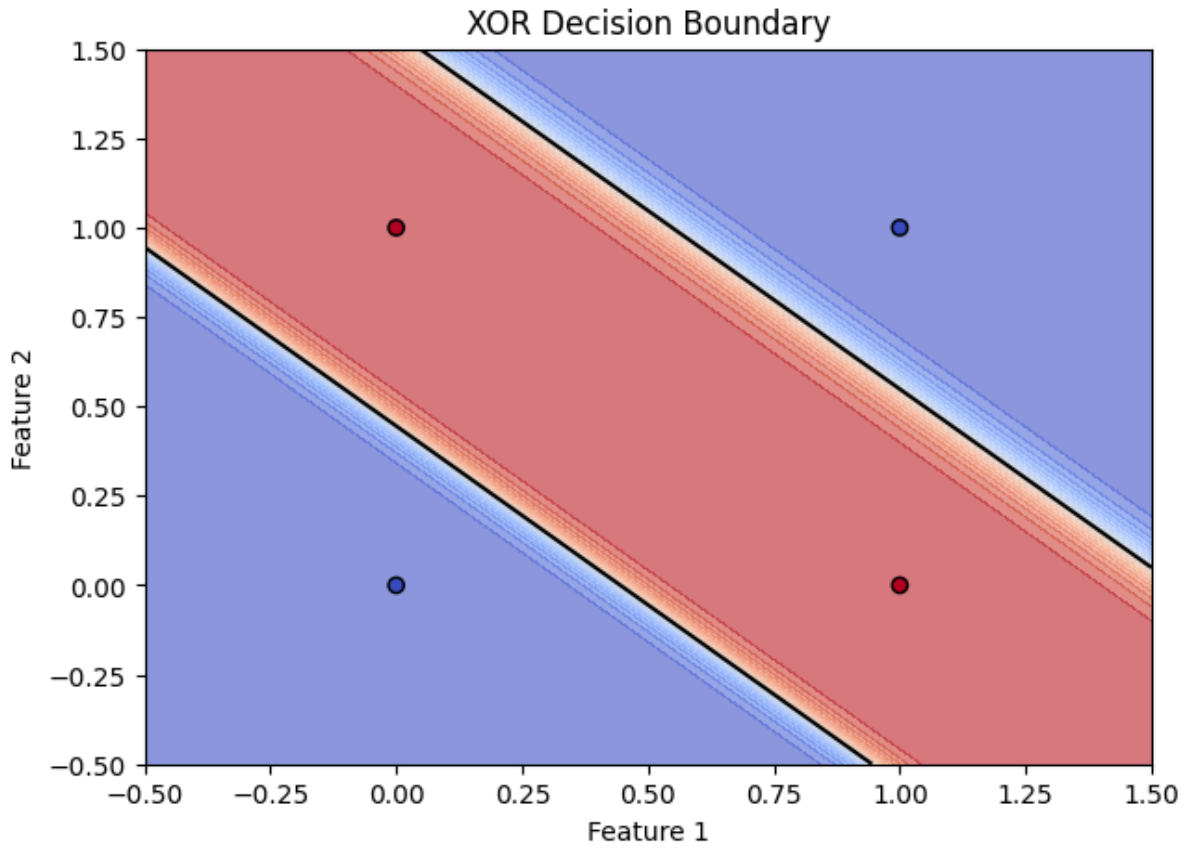
OUTPUT

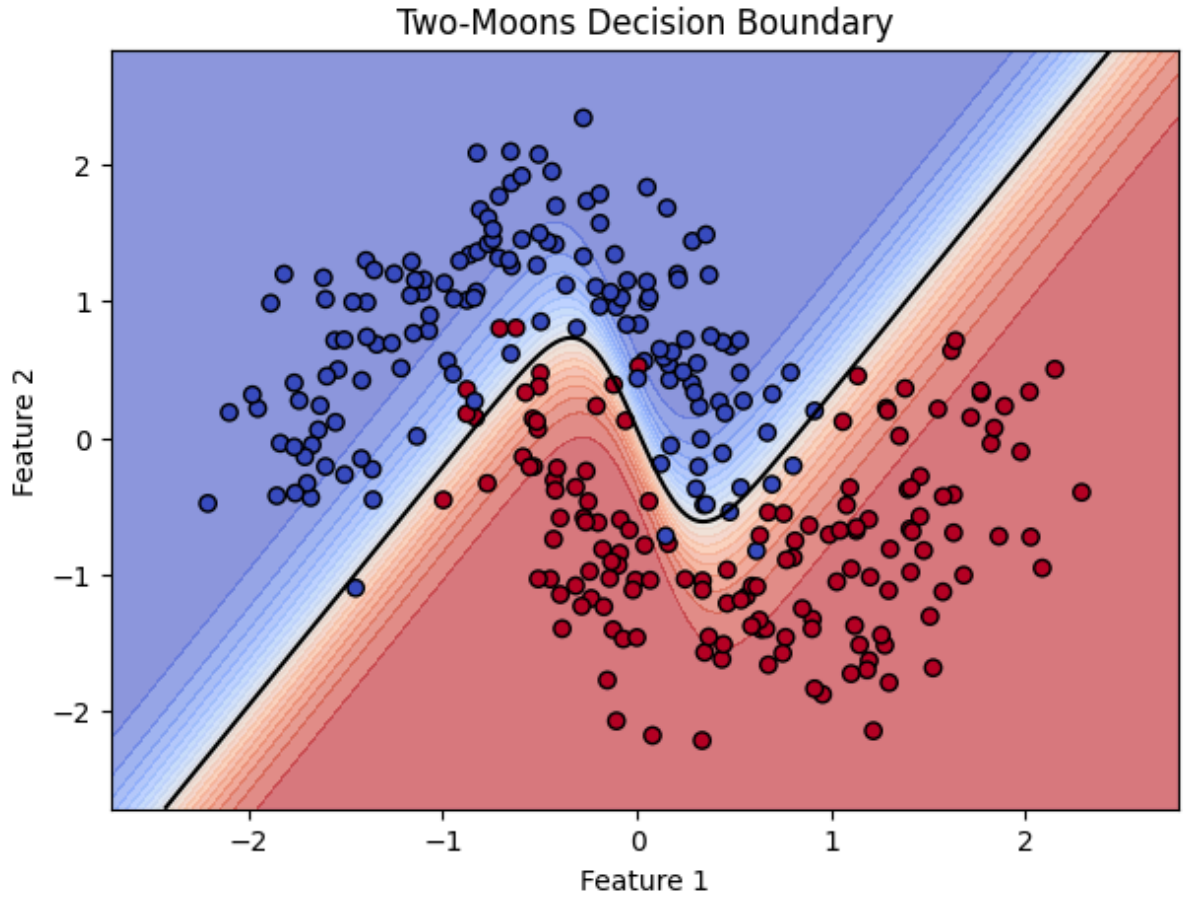
XOR Results
Inputs:
[[0. 0.]
[0. 1.]
[1. 0.]
[1. 1.]]
Actual outputs: [0. 1. 1. 0.]
Predicted probabilities: [1.100e-03 9.993e-01 9.993e-01 6.000e-04]
Predicted classes: [0 1 1 0]
Final XOR loss: 0.000772



Two-Moons Results
Final loss: 0.1217
Training accuracy: 96.0 %




XOR Decision Boundary
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
Feature 2
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50
Feature 1


Two-Moons Decision Boundary
2
1
0
−1
−2
Feature 2
−2
−1
0
1
2
Feature 1

Gradient Verification
Analytical gradient: -4.290297362716346e-05
Numerical gradient: -4.290297286272531e-05
Absolute difference: 7.644381479275479e-13

5. RESULT

A two-layer neural network was implemented from scratch using NumPy. Forward propagation, sigmoid activation, binary cross-entropy loss, backpropagation, and gradient descent were implemented successfully. The network was evaluated on XOR and two-moons datasets, and the loss curves and nonlinear decision boundaries were plotted. Numerical gradient verification was used to check the correctness of the backpropagation implementation. The experiment demonstrates how a hidden layer enables neural networks to learn nonlinear relationships that a single perceptron cannot represent.